

HealthAI MSPR

Объяснение новых этапов: устойчивые внешние API, Redis rate limiting и fallback-механизмы

Документ: техническое пояснение на русском языке

Контекст: backend FastAPI, Mistral API, Airflow REST API, Redis rate limiting и Redis cache

Дата: июнь 2026

1. Общая цель изменений

В проект HealthAI MSPR добавлены три взаимосвязанных механизма, которые повышают надежность backend-части: устойчивые вызовы внешних API, ограничение нагрузки через Redis и fallback-поведение для продолжения работы сервиса даже при частичных сбоях.

Главная идея: пользователь не должен сразу видеть техническую ошибку только потому, что внешний сервис временно недоступен. Backend сначала пытается восстановиться самостоятельно, затем переходит в деградированный режим, а критичные маршруты защищаются от перегрузки.

Этап	Что добавлено	Где реализовано
Устойчивые внешние API	Timeout, retries, backoff, circuit breaker, обработка ошибок авторизации и ответа	<code>backend/app/external_clients.py</code>
Rate limiting Redis	Ограничение количества запросов по IP/пользователю, отдельные лимиты для login и chat	<code>backend/app/rate_limit.py</code> , <code>backend/app/main.py</code>
Интеллектуальный cache Redis	Короткий TTL для свежих данных Airflow и более длинный stale cache на случай сбоя	<code>backend/app/cache.py</code> , <code>external_clients.py</code>
Fallback	Ответ-заглушка для Mistral, деградированный режим Airflow, fail-open при падении Redis	<code>chat.py</code> , <code>admin.py</code> , <code>rate_limit.py</code>

2. Интеграция внешних API с управлением сбоями

Раньше route `/chat/` напрямую создавал HTTP-клиент и отправлял запрос в Mistral. Такой подход работал, но вся логика ошибок находилась в route, и ее было сложно переиспользовать для других сервисов. Теперь внешние вызовы вынесены в отдельный модуль `external_clients.py`.

2.1. Какие внешние API покрыты

- **Mistral API** - используется Chat IA для генерации ответа пользователю.
- **Airflow REST API** - используется административной страницей flux для получения DAG runs.

2.2. Timeout

Timeout ограничивает максимальное время ожидания ответа. Если внешний сервис зависает, backend не держит соединение бесконечно. Для Mistral используется переменная:

```
MISTRAL_TIMEOUT_SECONDS=20
```

Если Mistral не ответил за заданное время, это считается временной ошибкой. Backend может выполнить повторную попытку, а затем перейти к fallback.

2.3. Retries

Retry - это повторная попытка выполнить запрос, если ошибка выглядит временной. Например, внешний сервис может вернуть 429 из-за нагрузки или 503 при кратком техническом сбое.

Код / ошибка	Почему retry уместен
429 Too Many Requests	Поставщик временно ограничил частоту запросов.
500 , 502 , 503 , 504	Ошибка на стороне внешнего сервиса или gateway, часто временная.
Timeout / network error	Сеть или сервис могли кратко быть недоступны.

```
EXTERNAL_API_MAX_RETRIES=2
EXTERNAL_API_RETRY_DELAY_SECONDS=0.25
```

При EXTERNAL_API_MAX_RETRIES=2 backend делает один основной вызов и до двух дополнительных попыток. Между попытками используется увеличивающаяся задержка: сначала базовая, затем больше. Это называется backoff и помогает не добивать сервис, который уже испытывает нагрузку.

2.4. Circuit breaker

Circuit breaker защищает backend от бесконечных вызовов заведомо неисправного сервиса. Если несколько попыток подряд завершаются ошибкой, circuit breaker "открывается" и временно запрещает новые запросы к этому внешнему API.

```
EXTERNAL_API_CIRCUIT_FAILURES=3
EXTERNAL_API_CIRCUIT_RECOVERY_SECONDS=60
```

- 1. Backend вызывает внешний API.
- 2. Если вызовы успешны, счетчик ошибок сбрасывается.
- 3. Если подряд накопилось 3 ошибки, circuit breaker открывается.
- 4. В течение 60 секунд backend не тратит время на новые вызовы к этому API.
- 5. После окна восстановления circuit breaker разрешает новый пробный вызов.

Это особенно важно для Chat IA: если Mistral недоступен, backend быстро переходит к ответу de secours и не блокирует пользователей долгим ожиданием.

3. Управление нагрузкой через Redis rate limiting

Rate limiting ограничивает количество запросов за заданный промежуток времени. В HealthAI MSPR это нужно, чтобы защитить backend от brute force на login, чрезмерных вызовов Chat IA и общей перегрузки API.

3.1. Почему используется Redis

Redis подходит для rate limiting, потому что он быстрый, хранит счетчики с TTL и может использоваться всеми экземплярами backend одновременно. Если позже backend будет масштабирован на несколько контейнеров, лимиты останутся общими, а не локальными для каждого процесса.

3.2. Как строится ключ лимита

Middleware определяет актера запроса. Если пользователь авторизован, ключ включает ID пользователя и IP. Если пользователь не авторизован, используется IP.

Пример:
rate_limit:POST:/chat/:user:1:ip:127.0.0.1:29743821

Последняя часть ключа - номер временного окна. Благодаря этому Redis хранит счетчик только для текущего окна, а затем автоматически удаляет его по TTL.

3.3. Разные лимиты для разных маршрутов

Маршрут	Переменные	Причина
/login	RATE_LIMIT_LOGIN_LIMIT , RATE_LIMIT_LOGIN_WINDOW_SECONDS	Защита от перебора пароля и слишком частых попыток входа.
/chat	RATE_LIMIT_CHAT_LIMIT , RATE_LIMIT_CHAT_WINDOW_SECONDS	Chat IA дороже обычных CRUD-запросов, потому что вызывает внешний AI provider.
Остальные маршруты	RATE_LIMIT_DEFAULT_LIMIT , RATE_LIMIT_DEFAULT_WINDOW_SECONDS	Общая защита API от всплесков нагрузки.

3.4. Что происходит при превышении лимита

Если счетчик в Redis превышает допустимый лимит, middleware не пропускает запрос в route. Backend сразу возвращает 429 Too Many Requests .

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 20
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 30

{
  "detail": "Trop de requetes. Reessayez dans quelques instants."
}
```

Заголовок `Retry-After` говорит клиенту, сколько секунд подождать перед новой попыткой.

3.5. Почему `/health` исключен

Endpoint `/health` используется мониторингом и оркестрацией. Если его ограничивать, система мониторинга может ошибочно решить, что API недоступен. Поэтому health check проходит без rate limiting.

4. Интеллектуальное кеширование Redis

Redis теперь используется не только для rate limiting, но и как cache для данных Airflow. Это снижает количество внешних HTTP-вызовов и повышает устойчивость административной страницы flux.

4.1. Почему кешируется Airflow, а не Chat IA

Данные Airflow являются административными и относительно стабильными: список DAG runs можно безопасно переиспользовать в течение короткого времени. Conversation Chat IA, наоборот, может содержать персональный контекст пользователя, поэтому ее не стоит кешировать по умолчанию.

4.2. Fresh cache и stale cache

Тип cache	Переменная	Назначение
Fresh cache	<code>AIRFLOW_CACHE_TTL_SECONDS=60</code>	Короткое хранение свежих runs Airflow, чтобы не дергать Airflow при каждом открытии страницы.
Stale cache	<code>AIRFLOW_STALE_CACHE_TTL_SECONDS=900</code>	Более длинная резервная копия. Используется только если Airflow недоступен.

4.3. Сценарий работы cache

1. Backend получает запрос `/admin/flux`.
2. Сначала проверяется свежий Redis cache `external:airflow:dag_runs`.
3. Если fresh cache найден, backend возвращает его со статусом `airflow_status=cached`.
4. Если fresh cache отсутствует, backend вызывает Airflow API.
5. При успешном ответе данные сохраняются в fresh cache и stale cache.
6. Если Airflow недоступен, backend пытается вернуть stale cache.
7. Если stale cache найден, возвращается `airflow_status=stale`.
8. Если cache вообще нет, возвращается деградированный ответ `airflow_status=degraded`.

Такой cache называется интеллектуальным, потому что он не просто ускоряет ответы, а участвует в стратегии отказоустойчивости: свежие данные используются для производительности, stale-данные - для продолжения сервиса при сбое Airflow.

5. Fallback-механизмы для непрерывности сервиса

Fallback - это запасное поведение, которое включается, когда основная зависимость не работает. В проекте реализованы три разных fallback-сценария.

5.1. Fallback для Mistral / Chat IA

Если Mistral недоступен после retry-попыток, route `/chat/` не возвращает пользователю техническую ошибку `502`. Вместо этого вызывается `build_chat_fallback_answer()`.

```
try:
    answer = call_mistral_chat(messages)
except ExternalServiceAuthError:
    raise HTTPException(status_code=503, detail="API Mistral non configuree ou refusee")
except ExternalServiceResponseError:
    raise HTTPException(status_code=502, detail="Reponse Mistral invalide")
except Exception:
    answer = build_chat_fallback_answer()
```

Важно различать типы ошибок:

- Если отсутствует или отклонена API key, это проблема конфигурации. Backend возвращает `503`.
- Если ответ Mistral пришел, но формат некорректный, backend возвращает `502`.
- Если Mistral временно недоступен, пользователь получает понятный fallback-ответ.

5.2. Fallback для Airflow

Административная страница flux использует Airflow REST API, чтобы показать состояние DAG runs. Если Airflow недоступен, backend не ломает всю страницу. Вместо этого он возвращает локальные метаданные и признак деградации.

```
{
  "airflow_disponible": false,
  "airflow_status": "degraded",
  "flux_metadonnees": {
    "consommations": { "total": 120, "dernier_run": "2026-06-16" }
  }
}
```

Пользователь-администратор понимает, что Airflow временно не отвечает, но при этом интерфейс остается usable: можно видеть локальные данные и не получать пустую ошибку сервера.

5.3. Fail-open fallback для Redis

Redis нужен для защиты нагрузки, но он не должен становиться единой точкой отказа. Если Redis временно недоступен, middleware пропускает запрос и пишет warning в лог.

Такое поведение называется fail-open: лучше временно пропустить запросы без rate limiting, чем полностью остановить API из-за сбоя Redis.

6. Итоговый сценарий работы Chat IA

1. Пользователь отправляет сообщение на `POST /api/chat/`.
2. Redis middleware проверяет лимит для пользователя/IP и маршрута `/chat`.
3. Если лимит превышен, backend возвращает `429`.
4. Если лимит не превышен, route проверяет JWT и права на изображения.
5. Route формирует system prompt и историю сообщений.
6. `call_mistral_chat()` вызывает Mistral через устойчивый HTTP-клиент.
7. При временных ошибках выполняются retry-попытки.
8. При повторных сбоях circuit breaker открывается.
9. Если сервис недоступен, пользователь получает fallback-ответ.
10. Если ответ Mistral корректный, backend возвращает настоящий AI answer.

7. Конфигурация

Переменная	Назначение
<code>REDIS_URL</code>	Адрес Redis для rate limiting, например <code>redis://redis:6379/0</code> .
<code>RATE_LIMIT_ENABLED</code>	Включает или отключает middleware rate limiting.
<code>RATE_LIMIT_LOGIN_LIMIT</code>	Количество запросов login за окно времени.
<code>RATE_LIMIT_CHAT_LIMIT</code>	Количество запросов chat за окно времени.
<code>CACHE_ENABLED</code>	Включает или отключает Redis cache.
<code>AIRFLOW_CACHE_TTL_SECONDS</code>	TTL свежего cache для данных Airflow.
<code>AIRFLOW_STALE_CACHE_TTL_SECONDS</code>	TTL резервного stale cache, используемого при сбое Airflow.
<code>EXTERNAL_API_MAX_RETRIES</code>	Количество повторных попыток для внешних API.
<code>EXTERNAL_API_CIRCUIT_FAILURES</code>	Сколько ошибок подряд нужно для открытия circuit breaker.
<code>EXTERNAL_API_CIRCUIT_RECOVERY_SECONDS</code>	Сколько секунд circuit breaker остается открытым.
<code>MISTRAL_TIMEOUT_SECONDS</code>	Максимальное время ожидания ответа Mistral.

8. Краткий вывод

Добавленные механизмы делают backend более устойчивым. Retry и timeout помогают пережить краткие сетевые ошибки. Circuit breaker защищает систему от зависших или падающих провайдеров. Redis rate limiting снижает риск brute force и перегрузки. Redis cache уменьшает количество вызовов Airflow и дает stale-данные при сбое. Fallback-сценарии позволяют пользователю продолжить работу с платформой даже при сбоях Mistral, Airflow или Redis.

В результате HealthAI MSPR переходит от простой интеграции внешних сервисов к более production-ready архитектуре: контролируемая нагрузка, предсказуемые ошибки и деградированный режим вместо полного отказа.